

Arduino Cookbook : De la Théorie à la Pratique

Chapitre 8 : Sorties Physiques (Physical Output)

Présenté par : Dr B. DIOP

29 mars 2026

Référence : Michael Margolis, *Arduino Cookbook*, 2nd Edition, O'Reilly

8.0 Introduction : Animer le monde physique

De l'électronique à la mécanique

Jusqu'à présent, nous avons traité des données et affiché des informations. Les "sorties physiques" permettent à l'Arduino d'agir sur son environnement en produisant du mouvement.

Dans ce chapitre, nous allons apprendre à contrôler trois grands types d'actionneurs :

- **Les Servomoteurs** : Pour un positionnement angulaire précis (bras robotiques, directions).
- **Les Moteurs à Courant Continu (DC)** : Pour la vitesse et la propulsion (roues de robots, ventilateurs).
- **Les Moteurs Pas-à-Pas (Stepper)** : Pour une rotation continue avec un contrôle exact de la position (imprimantes 3D, CNC).

8.1 Les Servomoteurs (Principe de base)

Qu'est-ce qu'un servomoteur ? C'est un moteur couplé à un capteur de position (potentiomètre) et un petit circuit électronique. Il ne tourne pas indéfiniment, mais se positionne à un angle précis, généralement entre **0 et 180 degrés**.

Connexion standard (3 fils) :

- **Rouge** : Alimentation (5V ou alimentation externe si le servo est puissant).
- **Noir ou Marron** : Masse (GND).
- **Jaune, Blanc ou Orange** : Signal de commande (relié à une broche Arduino).

[Insérer le schéma de câblage d'un servomoteur à l'Arduino]

8.2 Contrôler la position d'un Servomoteur

Bibliothèque requise : L'IDE Arduino inclut nativement la bibliothèque `Servo.h` qui génère les impulsions PWM spécifiques requises par ces moteurs.

```
#include <Servo.h>

Servo monServo;  // Creation de l'objet servo

void setup() {
  monServo.attach(9); // Attache le servo sur la broche 9
}

void loop() {
  monServo.write(0);    // Va a la position 0 degre
  delay(1000);          // Attend que le moteur atteigne sa position

  monServo.write(90);   // Va a la position centrale (90 degres)
  delay(1000);

  monServo.write(180);  // Va a la position maximale (180 degres)
  delay(1000);
}
```

8.3 Contrôler un Servo avec un Potentiomètre (Knob)

Application : Créer une commande manuelle où la position du moteur suit exactement la position d'un bouton rotatif (potentiomètre).

```
#include <Servo.h>

Servo monServo;
const int potPin = A0; // Broche du potentiometre

void setup() {
    monServo.attach(9);
}

void loop() {
    int lecturePot = analogRead(potPin); // Lecture de 0 a 1023

    // Convertit l'echelle 0-1023 vers l'echelle angulaire 0-180
    int angle = map(lecturePot, 0, 1023, 0, 180);

    monServo.write(angle); // Met a jour la position
    delay(15); // Petit delai pour la fluidite mecanique
}
```

8.4 Les Servomoteurs à rotation continue

Particularité : Certains servomoteurs sont modifiés en usine pour tourner à l'infini comme des moteurs classiques, tout en gardant l'électronique de contrôle d'un servo.

Changement de logique pour `servo.write(angle)` :

- `write(0)` : Tourne à vitesse maximale dans un sens.
- `write(180)` : Tourne à vitesse maximale dans l'autre sens.
- `write(90)` : **Arrêt du moteur** (point mort).

Idéal pour construire de petits robots mobiles à deux roues très facilement, car ils ne nécessitent pas de pont en H externe.

8.5 Les Moteurs à Courant Continu (DC Motors)

Limitation fondamentale : Un moteur DC standard (même petit) consomme beaucoup trop de courant (des centaines de mA, voire des Ampères au démarrage) pour être branché directement sur une broche de l'Arduino (limitée à 40mA).

Danger : Les courants induits (Flyback voltage)

Un moteur est une bobine. Lorsqu'on coupe son alimentation, il génère un pic de tension inverse très violent qui peut détruire l'Arduino. Il est **obligatoire** d'ajouter une **diode de roue libre** (ex : 1N4001) en parallèle du moteur.

8.6 Piloter un Moteur DC avec un Transistor

Solution matérielle : Utiliser l'Arduino pour commander un "interrupteur de puissance" : un transistor (Bipolaire NPN comme le TIP120, ou un MOSFET comme le IRLZ44N).

[Insérer le schéma : Arduino -> Résistance -> Base Transistor, Moteur + Diode]

Note : Le transistor permet d'allumer/éteindre et de varier la vitesse, mais **il ne permet pas de changer le sens de rotation.**

Code : Varier la vitesse du moteur DC

Une fois le transistor correctement câblé, la commande logicielle est identique à celle d'une LED via PWM (`analogWrite`).

```
const int moteurPin = 9; // Broche PWM reliee a la base/gate du transistor

void setup() {
  pinMode(moteurPin, OUTPUT);
}

void loop() {
  // Acceleration progressive
  for (int vitesse = 0; vitesse <= 255; vitesse++) {
    analogWrite(moteurPin, vitesse);
    delay(20);
  }

  // Deceleration progressive
  for (int vitesse = 255; vitesse >= 0; vitesse--) {
    analogWrite(moteurPin, vitesse);
    delay(20);
  }
}
```

8.7 Contrôler le sens de rotation : Le Pont en H (H-Bridge)

Problème : Comment inverser le courant pour faire reculer un robot, sans inverser les fils physiquement ?

Solution : Le Pont en H. C'est un arrangement de 4 interrupteurs (transistors) qui permet de diriger le courant dans les deux sens à travers le moteur.

[Insérer un schéma conceptuel des 4 interrupteurs du Pont en H]

8.8 Utiliser un circuit intégré L293D / L298N

Pratique : Au lieu de fabriquer un pont en H soi-même, on utilise des circuits intégrés dédiés (Motor Drivers) très robustes. Le **L293D** (ou son grand frère le module L298N) peut contrôler le sens et la vitesse de **deux** moteurs DC indépendamment.

- **Broche Enable :** Pour le contrôle de vitesse (PWM).
- **Broches Input 1 & Input 2 :** Pour la direction.
 - IN1=HIGH, IN2=LOW → Avance
 - IN1=LOW, IN2=HIGH → Recule
 - IN1=LOW, IN2=LOW → Arrêt libre

Code : Contrôle directionnel avec L293D

```
const int enablePin = 9; // Connecte a Enable 1 (PWM pour la vitesse)
const int in1Pin = 8;    // Connecte a Input 1 (Direction)
const int in2Pin = 7;    // Connecte a Input 2 (Direction)

void setup() {
  pinMode(enablePin, OUTPUT);
  pinMode(in1Pin, OUTPUT);
  pinMode(in2Pin, OUTPUT);
}

void loop() {
  // Vitesse maximale
  analogWrite(enablePin, 255);

  // Tourne dans le sens horaire
  digitalWrite(in1Pin, HIGH);
  digitalWrite(in2Pin, LOW);
  delay(2000);

  // Tourne dans le sens anti-horaire
  digitalWrite(in1Pin, LOW);
  digitalWrite(in2Pin, HIGH);
  delay(2000);
}
```

8.9 Les Moteurs Pas-à-Pas (Stepper Motors)

Principe : Contrairement au moteur DC qui tourne librement, le moteur pas-à-pas divise une rotation complète en un grand nombre d'étapes (pas). Par exemple, un moteur de 200 pas effectue 1,8 degré par pas.

Avantages :

- Contrôle exact de la position (sans capteur de retour).
- Excellent couple à basse vitesse.
- Capacité de maintien (Holding torque) : le moteur résiste à la rotation quand il est à l'arrêt mais alimenté.

8.10 Unipolaire vs Bipolaire

Moteurs Unipolaires (Ex : 28BYJ-48 avec module ULN2003) :

- Possèdent 5 ou 6 fils.
- Plus faciles à piloter car le courant circule toujours dans le même sens.
- Souvent utilisés avec le composant ULN2003 (réseau de transistors).

Moteurs Bipolaires (Ex : NEMA 17) :

- Possèdent 4 fils.
- Plus puissants, mais nécessitent un double Pont en H (ex : L298N ou contrôleur A4988) car il faut inverser la polarité dans les bobines.

[Insérer l'image comparant les fils d'un moteur unipolaire et bipolaire]

Code : Piloter un moteur Pas-à-Pas

L'IDE Arduino propose la bibliothèque native `Stepper.h` pour gérer l'activation séquentielle des bobines.

```
#include <Stepper.h>

const int pasParTour = 200; // Depend de la fiche technique du moteur
// Initialise le moteur sur les broches 8, 9, 10, 11
Stepper monMoteur(pasParTour, 8, 9, 10, 11);

void setup() {
    monMoteur.setSpeed(60); // Regle la vitesse a 60 tours par minute (RPM)
}

void loop() {
    // Fait un tour complet dans un sens (200 pas)
    monMoteur.step(pasParTour);
    delay(500);

    // Fait un demi-tour dans l'autre sens (-100 pas)
    monMoteur.step(-pasParTour / 2);
    delay(500);
}
```

Résumé du Chapitre 8

- **Alimentation** : Les moteurs consomment beaucoup. Ne les alimentez jamais directement depuis la broche 5V de l'Arduino. Utilisez une batterie ou une alimentation externe dédiée, en n'oubliant pas de **relier les masses (GND) ensemble**.
- **Sécurité (Diode)** : Une diode de roue libre est vitale pour tout moteur DC branché sur un transistor simple.
- **Servos = Simplicité** : Pour des mouvements angulaires simples (direction, bras), privilégiez les servomoteurs qui intègrent leur propre contrôleur.
- **Steppers = Précision** : Pour un contrôle millimétrique (imprimante 3D), le moteur pas-à-pas est la seule solution viable.

Fin du Chapitre 8. Voulez-vous que nous passions aux Sorties Audio (Chapitre 9) ou aux Télécommandes et Communications Sans Fil ?